

基本概念：ウィジェットはどのように機能するのか？

Flutter で Key-Value 形式を理解する

a. Key-value 形式は何か？

- Flutter では、ウィジェットのプロパティはキーと値のペアで定義される。
- キー → プロパティを表す（例：style,color）。
- 値 → プロパティの動作を決める（例：Colors.blue,TextStyle）。
- 例として Text ウィジェットに使用：

```
Text(  
  'Hello, Flutter!',      // This is a value (text being  
displayed)  
  style: TextStyle(      // 'style' is a key  
    fontSize: 20,        // 'fontSize' is a key, and '20' is the  
value  
    color: Colors.blue,  // 'color' is a key, and 'Colors.blue'  
is the value  
  ),  
);
```

Hello, Flutter!

b. なぜ Key-value 形式を使用？

- ウィジェットのプロパティをカスタマイズ → UI の外観や動作を簡単に制御できる。
- デフォルト値を上書きできる → 色,サイズ,配置などを自由に変更可能。
- 可読性が向上 → 名前付きパラメータにより,コードが明確になる。

c. いつ key のデフォルト value を使用？

- 値を指定しない場合,Flutter はデフォルト値を適用する。
- 別の動作をさせたいときは,デフォルトを上書きする。
- 例：デフォルトのテキストカラーは黒 → color: Colors.blue に変更可能。
- 例として Text ウィジェットにデフォルトを上書きする：

```
Container(  
  width: 200,  
  height: 100,  
  color: Colors.blue, // This sets the background color to  
blue  
  child: Center(  
    child: Text(  

```

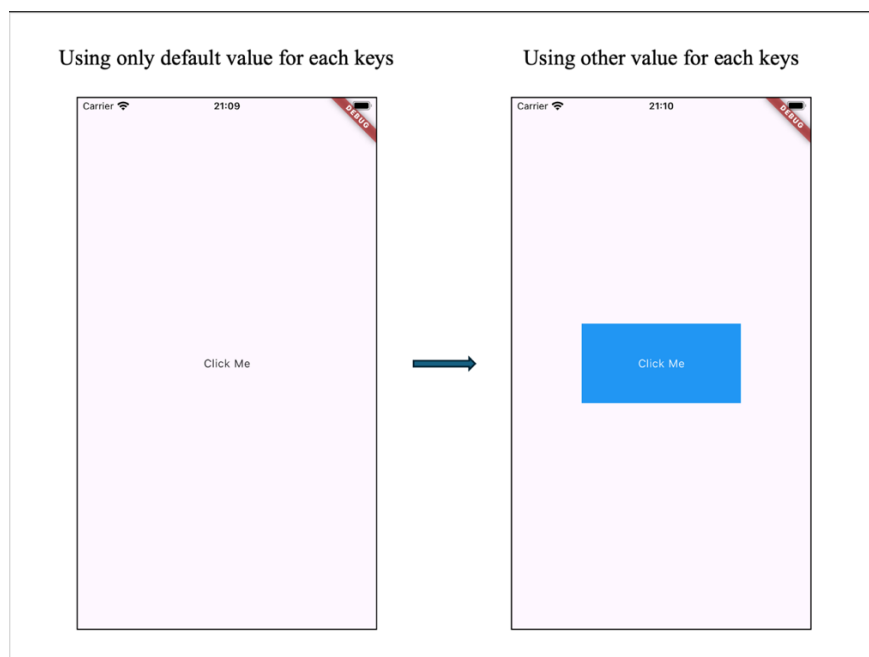
```

        'Click Me',
        style: TextStyle(color: Colors.white), // Text color
        is white
    ),
),
)

```

■ 説明：

- ◇ デフォルトでは,Container に背景色はない.
- ◇ `color: Colors.blue` を設定すると,背景色が青になる.
- ◇ `Text('Click Me')` は中央に配置され,白色で表示される.



d. Key-value 形式を Flutter で使用 vs JSON で使用

- Flutter と JSON はどちらもキーと値のペアを使うが,目的が異なる.
- Flutter → アプリの UI を定義する.

```

Text (
  "Hello, World!",
  textAlign: TextAlign.center,
)

```

- JSON → データの保存や転送を行う.

```

{
  "text": "Hello, World!",
  "textAlign": "center"
}

```

```
}
```

- 主な違い:

特徴	Flutter コード	JSON
目的	UI の構築と動作	データの保存と転送
実行方法	アプリ内で動作	データのやり取りに使用
例	<code>Text('Hello')</code>	<code>{ "text": "Hello" }</code>

Flutter でウィジェットがネストされる理由

a. ウィジェットネストとは？

- ウィジェットのネストとは、あるウィジェットを別のウィジェットの中に配置すること。
- これにより、UI 要素を組み合わせでより複雑なデザインを作成できる。
- Flutter の UI は、ネストされたウィジェットの階層で構成されている。
- 例として `Text` ウィジェットが `Button` ウィジェットの中にネストされる：

```
ElevatedButton(  
  onPressed: () {  
    print('Button clicked!');  
  },  
  child: Text('Click Me'), // Text is inside the Button  
);
```

b. なぜウィジェットネストを使用？

- 関連する UI 要素をまとめる → コードの可読性が向上。
- デザインを柔軟にカスタマイズできる → スタイルやレイアウトの調整が簡単。
- 再利用がしやすい → 繰り返し使えるコンポーネントを作成可能。
- 例として `Text` ウィジェットが `Container` ウィジェットの中にネストされる：

```
Container(  
  padding: EdgeInsets.all(10),  
  child: Text('This is a message'),  
);
```

c. どのようにネストはカスタムに役立つ

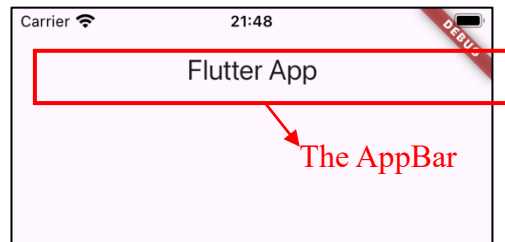
- ウィジェットを他のウィジェットの中に配置することで、外観を変更できる。
- 例えば、`Container` を使えばパディング、マージン、背景のスタイルを追加できる。

ステップ3：MyAppとウィジェットツリーを理解する

a. MyAppとは何か？

- `runApp()` を呼び出した後、アプリは **MyApp** ウィジェットをロードする。
- **MyApp** は Flutter アプリのルートウィジェット。
- すべての Flutter アプリはウィジェットで構成され、UI を形成する。
- これは基本的な **MyApp** の実装例：

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(  
          title: Text('My Flutter App'),  
        ),  
        body: Center(  
          child: Text('Hello, world!'),  
        ),  
      ),  
    );  
  }  
}
```



- 説明:
 - ◇ **MyApp** → アプリのエントリーポイントとして機能するカスタムウィジェット。
 - ◇ `StatelessWidget` → **MyApp** は状態を持たない（ステートレス）。
 - ◇ `build()` メソッド → UI の表示方法を定義する。
 - ◇ ウィジェットの階層構造（ウィジェットツリー）を返す。

b. ウィジェットツリーの仕組み

- Flutter の UI はツリー構造（ウィジェットツリー）で作られている。
- 各ウィジェットは子ウィジェットを持つことができ、階層を形成する。
- ウィジェットツリーはウィジェット同士の構造と関係性を定義する。
- これは **MyApp** 内のウィジェットツリーの例：

```
MaterialApp(  
  home: Scaffold(  

```

```

    appBar: AppBar(
      title: Text('My Flutter App'),
    ),
    body: Center(
      child: Text('Hello, world!'),
    ),
  ),
);

```

■ ツリー構造：

```

MaterialApp
├── Scaffold
│   ├── AppBar
│   │   └── Text ('My Flutter App')
│   └── Body
│       └── Center
│           └── Text ('Hello, world!')

```

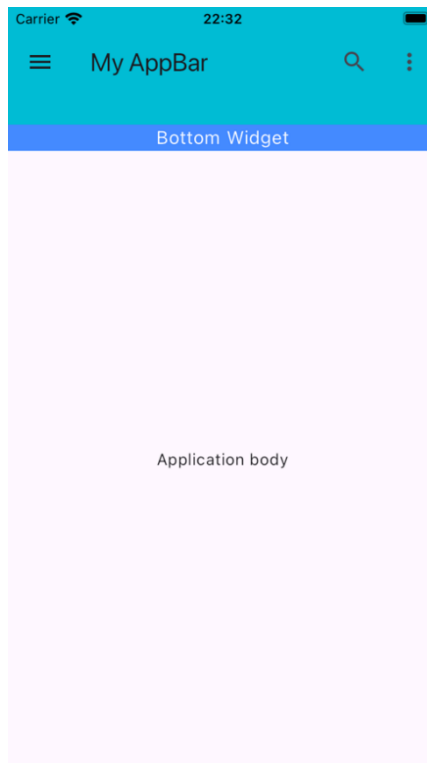
■ 説明:

- ◇ MaterialApp → すべてを包むルートウィジェット。
- ◇ Scaffold → アプリの基本構造を提供する（AppBar, body など）。
- ◇ AppBar → アプリ上部のバー（タイトルやアクション用）。
- ◇ Text → 画面にテキストを表示するウィジェット。

c. ツリー内の各ウィジェットの理解

ウィジェット	使用目的	例
MaterialApp	テーマやナビゲーションを設定するルートウィジェット	MaterialApp(home: Scaffold(...))
Scaffold	画面の基本レイアウト（AppBar や body など）を提供する	Scaffold(appBar: ..., body: ...)
AppBar	上部ナビゲーションバーを表示する	AppBar(title: Text('My App'))
Center	子ウィジェットを画面の中央に配置する	Center(child: Text('Hello'))
Text	シンプルなテキストを表示する	Text('Hello, world!')

- Flutter の各ウィジェットは特定の機能を持ち,他のウィジェットの中にネストできる。
- このネスト構造により,UI を効率的に整理できる。



d. AppBar を使うべき場面

- AppBar がよく使われる場面:
 - ◇ アプリの統一感のあるトップバーが必要なとき.
 - ◇ タイトルやナビゲーションボタンを追加したいとき.
 - ◇ 検索,設定,その他のオプションを提供したいとき.
 - ◇ タブや追加コンテンツを AppBar の下に表示したいとき.

Flutter の Text ウィジェット

a. Text ウィジェットとは？

- Flutter アプリでテキストを表示するウィジェット.
- プレーンテキストとスタイル付きテキストの両方をサポート.
- 静的メッセージ,動的更新,レスポンスデザインに対応可能.

b. 主なプロパティ

- 主なプロパティ：

プロパティ	説明	使用例
data	表示するテキスト（必須）.	'Hello, Flutter!'
style	テキストの見た目を変更.	TextStyle(fontSize: 18, color: Colors.blue)

textAlign	テキストの配置(left, center, right).	TextAlign.center
maxLines	最大行数を制限.	maxLines: 2
overflow	テキストのオーバーフロー処理 (ellipsis, fade).	TextOverflow.ellipsis
softWrap	テキストの折り返し設定.	softWrap: true

c. 例：FlutterでのTextスタイルの設定

- 例のFlutterコード：

```
Text(
  'Welcome to Flutter!', // The text to display
  style: TextStyle(
    fontSize: 20,          // Sets the font size
    color: Colors.green,   // Text color
    fontWeight: FontWeight.bold, // Makes the text bold
  ),
  textAlign: TextAlign.center, // Centers the text
  maxLines: 2,                // Limits to 2 lines
  overflow: TextOverflow.ellipsis, // Adds "..." if the
text is too long
)
```

- 動作説明:
 - ◇ 'Welcome to Flutter!' → 表示するテキスト.
 - ◇ style → サイズ,色,太さなどを指定.
 - ◇ textAlign: TextAlign.center → テキストを中央揃え.
 - ◇ maxLines: 2 → 最大2行までに制限.
 - ◇ overflow: TextOverflow.ellipsis → 長すぎる場合“…”を表示.

d. なぜTextウィジェットを使うのか？

- Textウィジェットを使う理由:
 - ◇ 静的・動的テキストの表示が簡単.
 - ◇ 整列・折り返し・オーバーフロー処理が可能.
 - ◇ どんなレイアウトやUIデザインにも適用できる.

FlutterのContainerウィジェット

a. Containerウィジェットとは？

- スタイル,位置,レイアウトを設定できるカスタマイズ可能な「ボックス」.
- 他のウィジェットをラップし,間隔,色,装飾を適用するのによく使われる.
- Flutterで最も柔軟で一般的に使用されるウィジェットの1つ.